Desk Number _____

Student Number _____

Student Name _____

School of Science and Engineering

MIDTERM EXAMINATION

Term 2, 2024

CSC1001 Introduction to Computer Science

Examination Duration: 120 minutes

Reading Time: 10 minutes

This examination has __5__ questions.

The score distribution of the 5 question is Question 1(40%), Question 2(20%), Question 3(10%), Question 4(15%), Question 5(15%)

Exam Conditions:

This is a FORMAL Examination

This is a RESTRICTED OPEN BOOK Exam. Maximum of one (1) sheet of handwritten notes double sided are permitted

Materials Permitted In The Exam Venue:

Maximum of one (1) sheet of handwritten notes double sided are permitted. **NO OTHER MATERIALS PERMITTED**

Any calculators without the functionalities of programming and file storage are permitted.

Materials To Be Supplied To Students:

1 × 6 Page Answer Booklet

You must write all your answers in your answer book.

Question 1. (40%)

Please describe what will be printed for each program as below.

	Program	Printed Results

1) 3%	`x = (1+8%5)//3 + int("34".replace("3",""))`
	`print(x)`
	
	Explanation: 8 % 5 is 3, so (1 + 3) // 3 is 1. Replacing 3 with an empty string in "34" gives "4", so the expression is 1 + 4 = 5.
		5
		
		3 分

2) 3%	`s1, s2 = "51+12", str(51+12)`
	`print(eval(s1+s2))`
	
	Explanation: s1 is the string "51+12" and s2 is "63". Concatenation gives "51+1263", which eval evaluates as 51 + 1263 = 1314.
		1314
		
		3 分

3) 3%	`t = [9, 42, "cuhksz", True, 7]`
	`print(t[1:3], t[2][1:3])`
	
	Explanation: t[1:3] selects items at indices 1 and 2, yielding [42, "cuhksz"]. The slice t[2][1:3] selects characters u and h.
		[42, 'cuhksz'] uh
		
		各 1 分

4) 3%	`x = 10`
	`if x==3 or 1:`
	`  print("csc1001")`
	
	`if x<2:`
	`  print("smaller than 2")`
	`elif x<=10:`
	`  print("smaller than 10")`
	`else:`
	`  print("others")`
	
	Explanation: x == 3 or 1 is true because 1 is truthy, so "csc1001" prints. Then x < 2 is false and x <= 10 is true, so "smaller than 10" prints.
		csc 1001 **(1 分)**
		
		smaller than 10 **(2 分)**

5) 3%	`n = 9`
	`factor = 2`
	
	`while factor <= n:`
	`  if n%factor == 0:`
	`  print(factor)`
	`  n/=factor`
	`  else:`
	`  factor+=1`
		3 **(1 分)**
		
		3 **(2 分)**

	Explanation: The first divisor found is 3, so it prints 3 and integer-divides n from 9 to 3. It prints 3 once more, reduces n to 1, then exits.	

6) 3%	`def f(a, b):`
	`  a = 3`
	`  b[2] = 3`
	`  return a`
	
	`a = 5`
	`b = [3, 4, 5, 6]`
	`print(f(a, b), b)`
	
	Explanation: Inside f, local a becomes 3 and the shared list is mutated at index 2. The function returns 3; afterward b is [3, 4, 3, 6].
		3 [3,4,3,6]
		1 分 +2 分

7) 4%	`s = "Jerry_20241110_CSC1001.txt"`
	`news = ""`
	
	`flag = False`
	`for c in s:`
	`  if c=="_" and flag == False:`
	`  flag = True`
	`  continue`
	
	`  if c=='_' and flag == True:`
	`  break`
	
	`  if flag:`
	`  news+=c`
	
	`print(s[6:14])`
	`print(news[:4])`
	
	Explanation: The first underscore sets flag to True and is skipped. Characters are accumulated up to the next underscore, which ends the loop. s[6:14] is "20241110" and news[:4] is "2024".
		20241110 **(1 分)**
		2024 **（2 分）**

8) 4%	`for x in [3, 5]:`
	`  for y in [x+1, 6]:`
	`  print(x, y)`
	
	Explanation: For x=3, y iterates through 4, 5, 6. For x=5, y iterates through 6, 7, 8. The nested loops print each pair on a separate line.
		3 4 **各 1 分**
		3 6
		5 6
		5 6

9) 3%	`s = "From hanxiaoguang@cuhk.edu.cn at Oct. 31st"`
	`print(s.split()[1].split(".")[0].split("@")[1])`
	
	Explanation: Whitespace splitting makes the email address the second word. Splitting it at the period
		cuhk
		3 分

	selects hanxiaoguang@cuhk; splitting that at @ and taking index 1 gives cuhk.	
10) 4%	d = [] d.append('apple') d.append(100) d.append('banana') print(d) d[2]+=str(d[1])+d[0] print(d) d[2].replace('a','$') print(d) Explanation: The first print shows the initial list. += appends the string 100apple to banana. replace() returns a new string, but its result is not assigned, so the final list stays unchanged.	['apple', 100, 'banana'] （1 分） ['apple', 100, 'banana100apple'] （1 分） ['apple', 100, 'banana100apple'] （2 分）
11) 3%	c = dict() c['apple'] = 15 c['banana'] = 30 for k in c: print(c[k]) c['apple'] = 45 print(c.get('Apple',4.5)) Explanation: The dictionary loop prints values 15 and 30. "Apple" differs from the key "apple", so get returns the default value 4.5.	15 30 4.5 各 1 分
12) 4%	t = {'a':20, 'b':2, 'c':22} tmp = [] for k, v in t.items(): tmp.append((v, k)) print(tmp) tmp.sort(reverse=True) print(tmp[0][1], tmp[1][1]) Explanation: The items are sorted in reverse tuple order by value: (22,"c"), (20,"a"), (2,"b"). The last expression prints the first two keys, c a.	[(20, 'a'), (2, 'b'), (22, 'c')] （2 分） c a （各 1 分）

Question 2. (5*4% = 20%)

Please answer the following short questions.

1) Please provide the Octal and Hexadecimal version of the binary number (11101110.0111).

Octal - 356.34; Hex - EE.7 各 2 分

Explanation: Group binary digits in threes around the radix point: 011 101 110 . 011 100 = octal 356.34. Grouping in fours gives 1110 1110 . 0111 0000 = hexadecimal EE.70, also written EE.7.

2) Please explain why python is a high-level programming language.

High development efficiency, easier to understood by programmers. (酌情给分)

Explanation: Python has readable high-level constructs and handles many low-level details, improving development speed and making code easier to understand.

3) Please describe the differences between arguments and parameters.

Arguments- 实参； parameters - 形参 （各 2 分）

Explanation: Parameters are names in a function definition; arguments are the actual values or expressions supplied when the function is called.

4) Use an example to explain the differences between "**break**" and "**continue**".

Break stop the loop; continue stop the iteration (各 2 分)

Explanation: break exits the nearest loop immediately. continue skips the rest of the current iteration and starts the next iteration.

5) Providing two ways to read the text from a file.

for line in fhand; fhand.read() (各 2 分)

Explanation: for line in fhand reads one line at a time; fhand.read() reads the remaining file contents as one string.

Question 3. (10%)

Given a list of valid passwords which are stored in **keys** (a list). Each password is in string type. Please write a program to instruct users to input a password until it matches one of the strings in **keys**. If the user enters more than 5 times, you should end the program and print a warning.

while True - 2 分

input 函数 - 2 分

Check input in keys - 2 分

Break - 2 分

判断超过 5 次 - 2 分

Explanation and sample solution: Keep an attempt counter, read a password on each pass, and test membership in keys. Stop on a match; after five failed attempts, print a warning.

```
for attempt in range(5):
    password = input("Enter password: ")
```

```
    if password in keys:
        print("Access granted")
        break
else:
    print("Warning: too many attempts")
```

Question 4. (15%)

Please first explain the usage of split() function, and then implement a function "def mySplit(s, a)": s is a string and a is a single character (in string type). Similar to the split() function, the output of the function is a list of sub-strings. (e.g., if s is "apple" and a is 'p', the result is ['a'‘','le'])

解释 split 的用法 - 3 分

for 循环 - 2 分

check == a - 2 分

累加 substring - 3 分

append to a new list - 2 分

考虑 empty - 2 分

return - 1 分

Explanation and sample solution: split(a) separates a string at each occurrence of delimiter a and returns a list of pieces. Scanning left to right and appending the collected segment at each delimiter preserves empty pieces, including at the beginning, end, or between consecutive delimiters.

```
def mySplit(s, a):
    parts = []
    current = ""
    for ch in s:
        if ch == a:
            parts.append(current)
            current = ""
        else:
            current += ch
    parts.append(current)
    return parts
```

Question 5. (15%)

You are given a file (score.txt) which is of many lines. For each line, it consists of one student name and her/his score (e.g., Jerry 95). Please write a program to read them and output the ranked name (in a descending order) into a new file (rank.txt): one name per line.

open file - 1 分

read - 2 分

take name and score out - 3 分

form a list of tuple type - 3 分

sort - 3 分

save to file - 3 分

Explanation and sample solution: Read each line, split it into the student's name and score, and store (score, name) pairs. Sort by score descending, then write each name to rank.txt in that order.

```
records = []
with open("score.txt") as source:
   for line in source:
      name, score = line.split()
      records.append((int(score), name))
records.sort(reverse=True)
with open("rank.txt", "w") as output:
   for score, name in records:
      output.write(name + "\n")
```

END OF EXAMINATION